



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251211
Nama Lengkap	Jendri Mansamnaf Japen
Minggu ke / Materi	14 / Tipe Data Set

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

MATERI 14.1

14.1.1 Pengenalan dan Mendefinisikan Set

Set adalah salah satu tipe data pada Python yang dapat digunakan untuk menyimpan sekumpulan data yang semuanya unik. Biasa juga dikenal dengan istilah himpunan. Beberapa sifat Set pada Python adalah:

- Isi dari Set disebut sebagai anggota (member).
- Anggota dari Set harus bersifat immutable. Beberapa tipe data immutable pada Python: integer, float, string, tuple dan lain-lain. Dengan demikian list dan dictionary (mutable) tidak dapat dimasukkan ke dalam Set.
- Set sendiri bersifat mutable, artinya anda dapat menambah atau mengurangi isi dari sebuah Set. Karena itu Set tidak dapat dimasukkan ke dalam Set.

Untuk mendefinisikan Set, ada beberapa cara yang dapat dilakukan yaitu dengan menggunakan notasi {} dan fungsi set() seperti contoh berikut ini:

```
# dengan menggunakan {}
bilangan_genap = {2, 4, 6, 8, 10, 12}
bilangan_ganjil = {1, 3, 5, 7, 9, 11}

# dengan menggunakan fungsi set()
pernah_ke_bulan = set('Neil Armstrong', 'Buzz Aldrin')
```

Untuk mendefinisikan Set kosong anda tidak dapat menggunakan notasi {}, tetapi harus menggunakan fungsi set() seperti contoh berikut:

```
# dengan fungsi set()
pernah_ke_mars = set() # menghasilkan set kosong

data = {} # ini akan menghasilkan dictionary kosong
```

14.1.2 Pengaksesan Set

Set tidak memiliki indeks, karena itu anda tidak dapat mengakses anggota-anggota dari sebuah Set secara langsung menggunakan indeks. Perhatikan contoh program berikut ini:

```

nim = {'71200120', '71200195', '71200214'}
jumlah_nim = len(nim)
print(jumlah_nim)      # akan menghasilkan output 3

# tampilkan isi set satu-persatu
for n in nim:
    print(n)

```

Output yang dihasilkan dari program tersebut adalah sebagai berikut:

```

3
71200214
71200195
71200120

```

Jika dicermati, output yang dihasilkan urutannya berbeda dengan deklarasi Set sebelumnya. Hal ini disebabkan karena pada Set tidak ada indeks, sehingga tidak ada urutan posisi anggota. Pada tipe data Set, posisi anggota tidaklah penting. Set adalah tipe data mutable, artinya isinya bisa bertambah atau berkurang. Program berikut ini menunjukkan bagaimana cara menambah anggota sebuah Set dengan fungsi add():

```

# definisikan sebuah set kosong
plat_nomor = set()

# tambahkan plat nomor 'AB 1890 XA'
plat_nomor.add('AB 1890 XA')

# tambahkan plat nomor 'AD 6810 MT'
plat_nomor.add('AD 6810 MT')

# jumlah anggota di dalam Set
print(len(plat_nomor))

# tambahkan plat yang sama sekali lagi
plat_nomor.add('AB 1890 XA')

# tampilkan semua plat nomor
for plat in plat_nomor:
    print(plat)

```

Output yang dihasilkan adalah sebagai berikut:

```

2
AD 6810 MT
AB 1890 XA

```

Set memiliki mekanisme untuk melakukan pengecekan apakah anggota baru yang akan dimasukkan sudah ada di dalam Set (cek duplikasi). Jika belum ada, maka anggota tersebut bisa masuk ke dalam Set. Tetapi jika ternyata di dalam Set sudah ada anggota dengan nilai yang sama, maka pemanggilan fungsi add() tidak akan menambah anggota dari Set. Pengecekan duplikasi ini sudah ada di dalam fungsi add(), sehingga anda tidak perlu melakukannya sendiri. Untuk menghapus anggota dari sebuah Set, ada beberapa cara yaitu dengan fungsi discard(), remove(), pop() dan clear(). Perbedaan dari empat fungsi

tersebut dapat dilihat pada Gambar 12.1. Berikut ini adalah contoh program yang mendemonstrasikan penghapusan anggota dari sebuah Set:

```
bilangan_prima = {13, 23, 7, 29, 11, 5}

# hapus 5 dari set tersebut
bilangan_prima.remove(5)
```

discard()	remove()	pop()	clear()
Menghapus satu elemen yang disebutkan	Menghapus satu elemen yang disebutkan	Mengambil salah satu dan menghapusnya dari set (tidak tentu)	Menghapus seluruh elemen di dalam set
Tidak ada error	Muncul error jika elemen yang dihapus tidak ada	Error jika set kosong	Tidak ada error

Gambar 12.1: Fungsi-fungsi untuk menghapus anggota dari Set.

```
print(bilangan_prima)

# hapus 97 (tidak ada)
bilangan_prima.discard(97)
print(bilangan_prima)

# ambil dan hapus salah satu
bilangan = bilangan_prima.pop()
print(bilangan)
print(bilangan_prima)

# kosongkan set
bilangan_prima.clear()
print(bilangan_prima)
```

Output yang dihasilkan dari program tersebut adalah:

{5, 7, 11, 13, 23, 29} # awal

{7, 11, 13, 23, 29} # setelah 5 dihapus. Perhatikan urutan berubah

{7, 11, 13, 23, 29} # 97 tidak ada, sehingga Set tidak berubah

7 # fungsi pop() mengeluarkan 7

{11, 13, 23, 29} # setelah 7 keluar dari Set

set() # setelah isi dari Set dihapus semua

Fungsi discard() tidak akan menghasilkan error jika anggota yang ingin dihapus tidak ada di dalam Set. Sedangkan fungsi pop() akan mengambil salah satu anggota (secara acak) dan mengeluarkannya dari Set. Fungsi pop() akan berguna jika kita ingin memproses isi dari Set satu-persatu tanpa memperdulikan urutan/posisi dari setiap anggota di dalam Set. Bagaimana jika anda ingin mengubah nilai salah satu

anggota di dalam Set? Pada Set anda tidak bisa mengubah langsung nilai dari salah satu anggota di dalam Set. Yang dapat dilakukan adalah melakukan operasi penggantian (replace) dengan cara menghapus anggota yang mau diubah, kemudian memasukkan anggota baru yang nilainya sesuai dengan yang diinginkan. Contohnya dapat dilihat pada program berikut ini:

```
# Buat Set dari List
ikan = set(['koi', 'koki', 'kembung', 'salmon'])
print(ikan)

# ganti koi menjadi teri
ikan.remove('koi')
ikan.add('teri')
print(ikan)
```

Output yang dihasilkan dari program tersebut adalah sebagai berikut:

```
{'koi', 'koki', 'salmon', 'kembung'}
{'teri', 'salmon', 'kembung', 'koki'}
```

Untuk mengubah nilai 'koi' menjadi 'teri', yang perlu dilakukan adalah hapus dulu anggota 'koi', kemudian masukkan anggota baru dengan nilai 'teri'. Sekali lagi, yang perlu ditekankan di sini adalah pada Set tidak mengenal adanya posisi/urutan data. Sehingga output dari program tersebut nilai 'koi' sudah tidak ada, digantikan oleh nilai 'teri'. Kemudian urutan isi Set ikan saat ditampilkan tidaklah sama (ada perubahan). Setiap kali ada operasi penambahan dan penghapusan maka urutan anggota di dalam Set biasanya akan berubah.

14.1.3 Operasi-Operasi pada Set

Operasi-operasi pada Set adalah operasi-operasi pada himpunan. Berikut ini adalah daftar operasi-operasi Set pada Python:

- **Operator Union.** Menggabungkan dua Set menjadi satu. Dapat menggunakan operator | maupun fungsi union().
- **Operator Intersection.** Menghasilkan irisan dari dua Set. Dapat menggunakan operator & maupun fungsi intersection().
- **Operator Difference.** Menghasilkan Set baru yang merupakan selisih dari dua Set yang dibandingkan. Dapat menggunakan operator - maupun fungsi difference().
- **Operator Symmetric Difference.** Menghasilkan Set baru yang merupakan jumlah dari dua Set kecuali irisannya. Dapat menggunakan operator maupun fungsi symmetric_difference().

Ilustrasi dari operasi-operasi tersebut dapat dilihat pada Gambar 12.2.

Operator Union

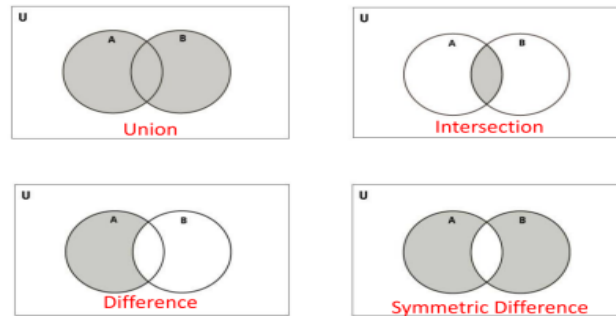
Contoh penggunaan operator union dapat dilihat pada contoh program berikut ini:

```

merek_hp = {'Samsung', 'Apple', 'Xiaomi', 'Sony'}
merek_ac = {'LG', 'Samsung', 'Panasonic', 'Daikin', 'Sony'}

# union dari merek_hp dan merek_ac
gabungan = merek_hp | merek_ac

```



Gambar 12.2: Operasi-operasi pada Set.

```

# bisa juga menggunakan gabungan = merek_hp.union(merek_ac)
print(gabungan)

```

Output yang dihasilkan dari program tersebut adalah anggota dari Set merek_hp digabungkan dengan anggota dari Set merek_ac, menghasilkan Set baru bernama gabungan. Ada duplikasi (anggota yang sama) yaitu 'Samsung' dan 'Sony'. Sehingga 'Samsung' dan 'Sony' di dalam Set gabungan hanya muncul 1 kali karena anggota dari Set harus unik. Outputnya adalah sebagai berikut:

```
{'Xiaomi', 'Sony', 'LG', 'Daikin', 'Samsung', 'Apple', 'Panasonic'}
```

Operator Intersection

Contoh penggunaan operator Intersection dapat dilihat pada program berikut:

```

renang = {'siti', 'mail', 'ikhshan', 'upin', 'ipin'}
tenis = {'joko', 'mail', 'ipin', 'upin', 'tejo'}

# suka renang dan tenis
renang_tenis = renang & tenis
print(renang_tenis)

```

Program tersebut akan menghasilkan output hasil operasi intersection dari Set renang dan tenis. Intersection berarti anggota-anggota yang berada di dalam Set renang dan Set tenis sekaligus, yaitu mail, upin dan ipin. Output yang dihasilkan dari program tersebut adalah:

```
{'upin', 'ipin', 'mail'}
```

Operator Difference

Contoh penggunaan operator difference dapat dilihat pada program berikut:

```

# bisa berbahasa english
english = {'desi', 'tono', 'evan', 'miko', 'takashi', 'chaewon'}

# bisa berbahasa korea
korean = {'chaewon', 'yeona', 'erika', 'miko'}

# siapa yang hanya bisa bahasa korea?
only_korean = korean - english
print(only_korean)

# siapa yang hanya bisa bahasa english?
only_english = english - korean
print(only_english)

```

Operator difference akan menghasilkan Set yang anggotanya merupakan selisih dari kedua Set yang dibandingkan. Pada contoh tersebut digunakan untuk mendapatkan anggota yang hanya bisa berbahasa korea dengan cara mencari selisih antara Set korean dan Set english. Sedangkan kebalikannya, jika ingin mengetahui siapa saja yang hanya bisa berbahasa English maka cari selisih antara set English dan Set Korea. Output yang dihasilkan dari program tersebut adalah sebagai berikut:

```

{'erika', 'yeona'} # hanya bisa Korea
{'takashi', 'desi', 'evan', 'tono'} # hanya bisa English

```

Operator Symmetric Difference

Contoh penggunaan operator symmetric difference dapat dilihat pada program berikut ini:

```

english = {'desi', 'tono', 'evan', 'miko', 'takashi', 'chaewon'}
korean = {'chaewon', 'yeona', 'erika', 'miko'}

# hanya bisa bicara satu bahasa saja
one_language = english ^ korean
print(one_language)

```

Operator symmetric difference akan menghasilkan Set baru yang merupakan gabungan dari dua Set tetapi tidak termasuk irisannya. Contoh yang diberikan memanfaatkan operator symmetric difference untuk mendapatkan siapa saja yang hanya dapat berbicara dalam satu bahasa (artinya tidak termasuk irisannya). Secara umum dapat juga dihasilkan dari operasi berikut:

```

one_language = english.union(korean) - english.intersection(korean)

```

Output yang dihasilkan dari program tersebut adalah:

```

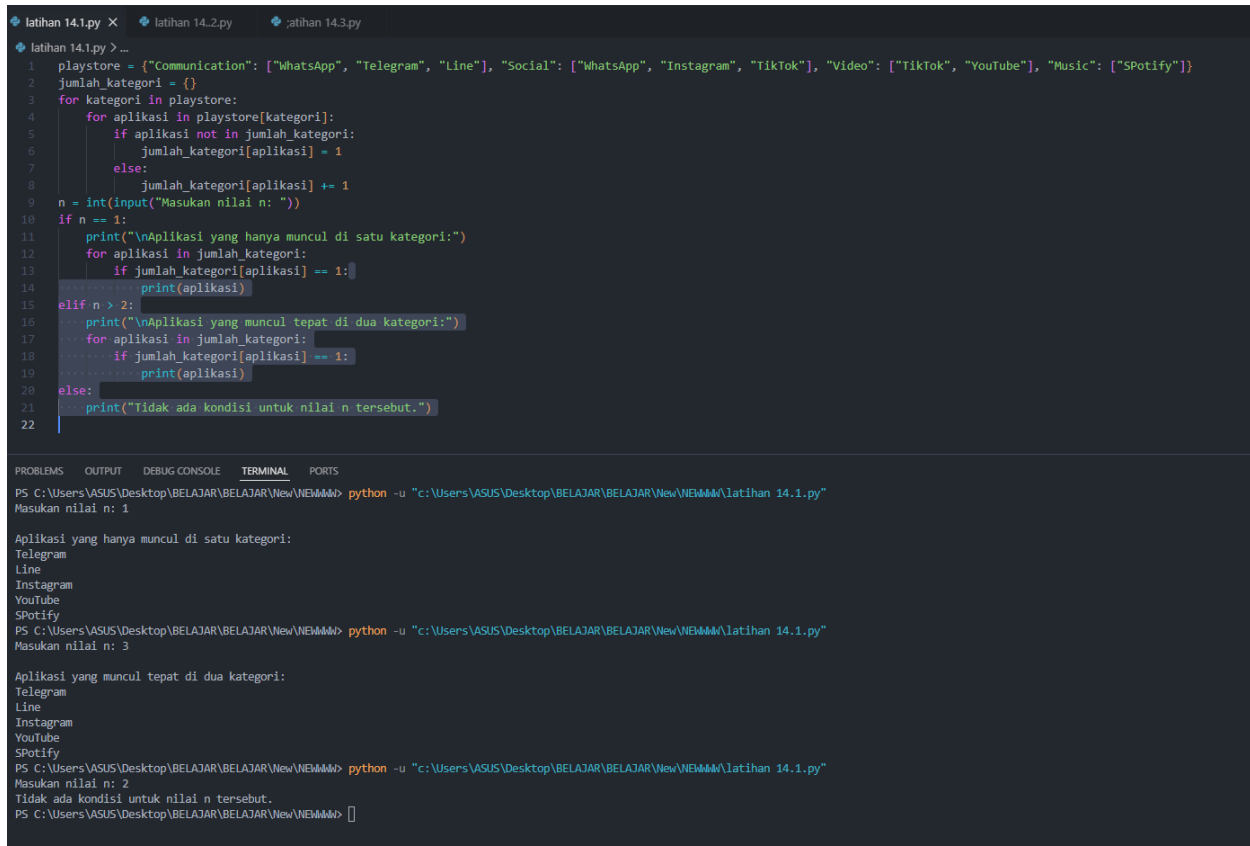
{'yeona', 'tono', 'desi', 'erika', 'evan', 'takashi'}

```

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Latihan 14.1



```
latihan 14.1.py x latihan 14.2.py latihan 14.3.py
latihan 14.1.py > --
1 playstore = {"Communication": ["WhatsApp", "Telegram", "Line"], "Social": ["WhatsApp", "Instagram", "TikTok"], "Video": ["TikTok", "YouTube"], "Music": ["SPotify"]}
2 jumlah_kategori = {}
3 for kategori in playstore:
4     for aplikasi in playstore[kategori]:
5         if aplikasi not in jumlah_kategori:
6             jumlah_kategori[aplikasi] = 1
7         else:
8             jumlah_kategori[aplikasi] += 1
9 n = int(input("Masukan nilai n: "))
10 if n == 1:
11     print("\nAplikasi yang hanya muncul di satu kategori:")
12     for aplikasi in jumlah_kategori:
13         if jumlah_kategori[aplikasi] == 1:
14             print(aplikasi)
15 elif n > 2:
16     print("\nAplikasi yang muncul tepat di dua kategori:")
17     for aplikasi in jumlah_kategori:
18         if jumlah_kategori[aplikasi] == 2:
19             print(aplikasi)
20 else:
21     print("Tidak ada kondisi untuk nilai n tersebut.")
22
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\VASUS\Desktop\BELAJAR\BELAJAR\New\NIEMMMM> python -u "c:\Users\VASUS\Desktop\BELAJAR\BELAJAR\New\NIEMMMM\latihan 14.1.py"

Masukan nilai n: 1

Aplikasi yang hanya muncul di satu kategori:

Telegram

Line

Instagram

YouTube

SPotify

PS C:\Users\VASUS\Desktop\BELAJAR\BELAJAR\New\NIEMMMM> python -u "c:\Users\VASUS\Desktop\BELAJAR\BELAJAR\New\NIEMMMM\latihan 14.1.py"

Masukan nilai n: 3

Aplikasi yang muncul tepat di dua kategori:

Telegram

Line

Instagram

YouTube

SPotify

PS C:\Users\VASUS\Desktop\BELAJAR\BELAJAR\New\NIEMMMM> python -u "c:\Users\VASUS\Desktop\BELAJAR\BELAJAR\New\NIEMMMM\latihan 14.1.py"

Masukan nilai n: 2

Tidak ada kondisi untuk nilai n tersebut.

PS C:\Users\VASUS\Desktop\BELAJAR\BELAJAR\New\NIEMMMM> []

Program digunakan untuk menghitung jumlah kategori dari setiap aplikasi di Play Store. Jika $n = 1$, program menampilkan aplikasi yang hanya ada di satu kategori. Jika $n > 2$, program menampilkan aplikasi yang muncul tepat di dua kategori.

Latihan 14.2

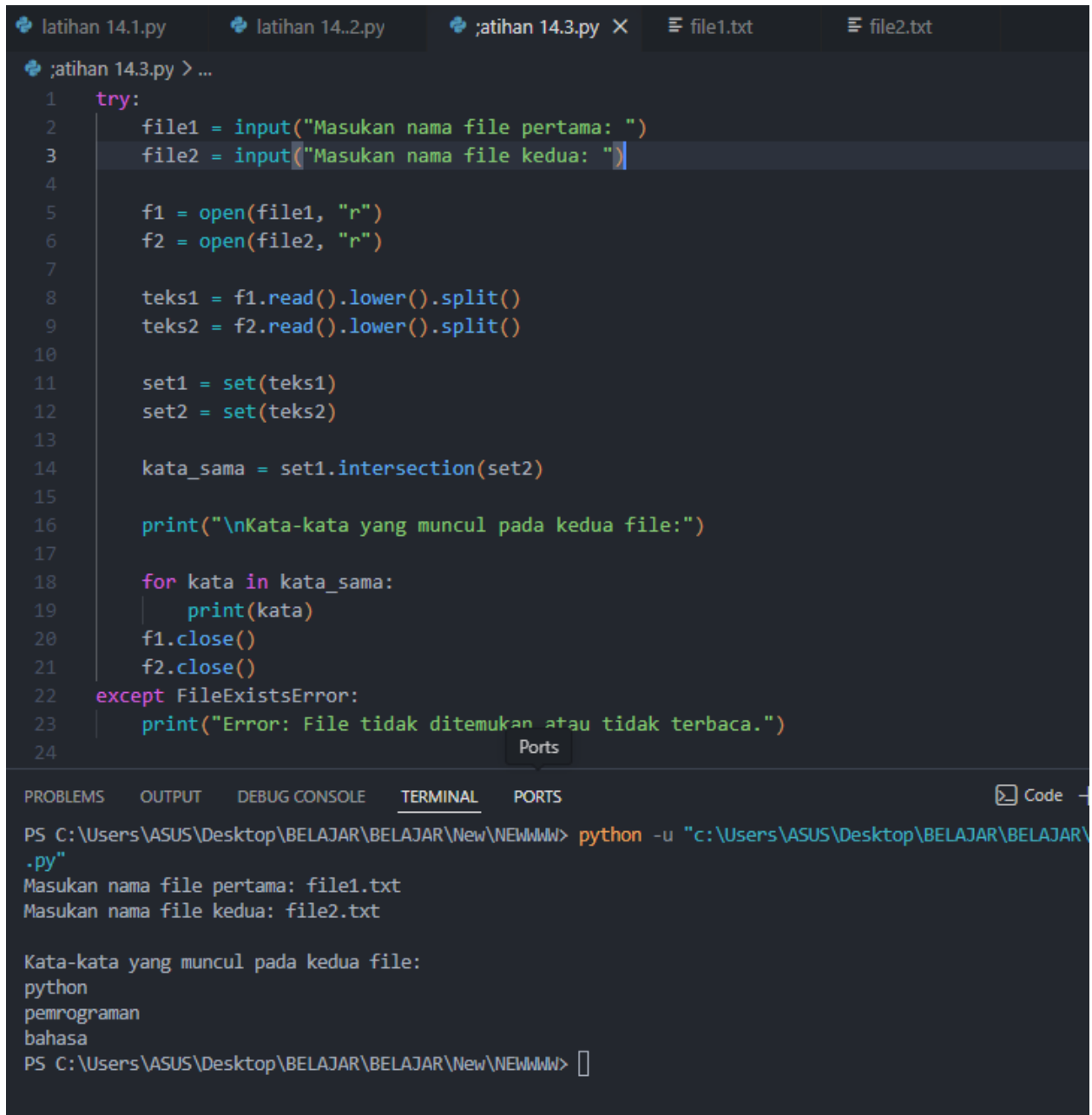
```
latihan 14.1.py  latihan 14.2.py X  latihan 14.3.py
latihan 14.2.py > ...
7  data_set = {10, 20, 30, 40}
8  print("Set sebelum konversi\t:", data_set)
9  hasil_list = list(data_set)
10 print("List setelah konversi\t:", hasil_list)
11 print("-" * 30)
12
13 data_tuple = ("A", "B", "B", "c")
14 print("Tuple sebelum konversi\t:", data_tuple)
15 hasil_set2 = set(data_tuple)
16 print("Set setelah konversi\t:", hasil_set2)
17 print("-" * 30)
18
19 data_set2 = {"Python", "Java", "C++"}
20 print("Set sebelum konversi\t:", data_set2)
21 hasil_tuple = tuple(data_set2)
22 print("Tuple setelah konversi\t:", hasil_tuple)
23 print("-" * 30)
24
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\ASUS\Desktop\BELAJAR\BELAJAR\New\NEWWWW> python -u "c:\Users\ASUS\Desktop\BELAJAR\BELAJAR\New\NEWWWW\latihan 14.2.py"
List sebelum konversi : [1, 2, 2, 3, 4, 4, 5]
Set setelah konversi : {1, 2, 3, 4, 5}
-----
Set sebelum konversi : {40, 10, 20, 30}
List setelah konversi : [40, 10, 20, 30]
-----
Tuple sebelum konversi : ('A', 'B', 'B', 'c')
Set setelah konversi : {'c', 'A', 'B'}
-----
Set sebelum konversi : {'Java', 'Python', 'C++'}
Tuple setelah konversi : ('Java', 'Python', 'C++')
-----
PS C:\Users\ASUS\Desktop\BELAJAR\BELAJAR\New\NEWWWW> 
```

Program di atas digunakan untuk mendemonstrasikan proses konversi antar struktur data di Python, yaitu List, Set, dan Tuple. Program menampilkan isi data sebelum dan sesudah konversi sehingga perbedaan setiap tipe data dapat terlihat dengan jelas.

Latihan 14.3



```
latihan 14.1.py  latihan 14.2.py  ;atihan 14.3.py X  file1.txt  file2.txt
;atihan 14.3.py > ...
1  try:
2      file1 = input("Masukan nama file pertama: ")
3      file2 = input("Masukan nama file kedua: ")
4
5      f1 = open(file1, "r")
6      f2 = open(file2, "r")
7
8      teks1 = f1.read().lower().split()
9      teks2 = f2.read().lower().split()
10
11     set1 = set(teks1)
12     set2 = set(teks2)
13
14     kata_sama = set1.intersection(set2)
15
16     print("\nKata-kata yang muncul pada kedua file:")
17
18     for kata in kata_sama:
19         print(kata)
20     f1.close()
21     f2.close()
22 except FileNotFoundError:
23     print("Error: File tidak ditemukan atau tidak terbaca.")
24
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Code

```
PS C:\Users\ASUS\Desktop\BELAJAR\BELAJAR\New\NEW\NEW> python -u "c:\Users\ASUS\Desktop\BELAJAR\BELAJAR\
.py"
Masukan nama file pertama: file1.txt
Masukan nama file kedua: file2.txt

Kata-kata yang muncul pada kedua file:
python
pemrograman
bahasa
PS C:\Users\ASUS\Desktop\BELAJAR\BELAJAR\New\NEW\NEW> 
```

Program ini digunakan untuk membaca dua file teks dan mencari kata-kata yang muncul pada kedua file tersebut. Semua kata diubah menjadi lowercase agar perbandingan lebih akurat, dan program juga menangani error jika file tidak ditemukan atau tidak bisa dibaca.

Link github: <https://github.com/japenjendri-design/Praktikum-Algoritma-dan-Pemrograman.git>